

4주차 유실과 중복 중에서 무엇을 고를 것인가

4주차 · Kafka

이 장의 수치는 모두 `practice/chapter4/` 의 코드가 남긴 산출물에서 옮긴 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

3·4절의 수치는 Kafka 브로커가 떠 있어야 다시 만들 수 있으므로 **과거 실행이 남긴 산출물에서 옮긴 값**이다. 5·6절의 수치는 브로커 없이 지금 다시 실행해 얻은 로그에서 옮겼다.

- ☐ 3주차에서 파이프라인을 다섯 계층으로 나누고 수집 계층에 Kafka를 놓았음
- ☐ 그때 미뤄 둔 판단이 하나 있음 — **전달 보증 수준을 무엇으로 할 것인가**
- ☐ 이 장은 그 판단을 코드 한 줄의 위치로 바꿔 보고, 그 결과를 유실 건수와 중복 건수로 확인함
- ☐ 이 장도 설명과 실습을 나누지 않음
- ☐ 절마다 실행할 명령이 나오고 그 아래에 실제 산출물과 해석이 이어짐
- ☐ 3·4절은 Docker가 필요하고, 5·6절은 Python만으로 순서대로 실행함

1. 오늘의 질문

- ☐ 토픽에 넣은 순서는 어디까지 보존될까?
- ☐ 커밋 한 줄을 처리 앞에 두느냐 뒤에 두느냐가 무엇을 바꿀까?
- ☐ 유실과 중복 중 공공 데이터가 먼저 피해야 할 것은 무엇일까?
- ☐ 같은 이벤트가 두 번 도착해도 업무 상태가 한 번 반영된 것과 같으려면 무엇이 있어야 할까?

2. 이 장에서 실행할 것

- ☐ 소비자가 어디까지 읽었는지 기록하는 방식은 **책갈피**와 속성이 하나씩 대응함
- ☐ 책갈피가 꽂힌 쪽 번호 = 커밋된 오프셋, 한 쪽을 읽는 행위 = 이벤트 하나를 처리하는 일, 책을 갑자기 덮는 일 = 소비자 프로세스의 장애
- ☐ 책갈피를 **먼저** 옮겨 놓고 읽다가 책을 덮으면, 다음에 펼쳤을 때 읽지 않은 쪽부터가 아니라 그 뒤부터 읽게 됨 → 건너뛴 쪽이 생김
- ☐ 다 읽은 **뒤에** 책갈피를 옮기기로 했는데 옮기기 전에 책을 덮으면, 다음에 펼쳤을 때 이미 읽은 쪽을 다시 읽음
- ☐ 비유는 여기까지다
- ☐ 책은 한 권이라 책갈피도 하나지만, 토픽은 파티션으로 나뉘어 있어 책갈피가 파티션마다 따로 있음

- 그리고 같은 쪽을 두 번 읽으면 사람은 알아차리지만, 같은 이벤트를 두 번 처리한 시스템은 아무 표시도 남기지 않음
- 이 두 번째 차이가 5절에서 저장된 행 수로 나타남

시작 전 준비

- 3·4절은 실제 Kafka 브로커에 이벤트를 보내고 받으므로 Docker 데몬이 떠 있어야 함
- 다음 명령이 Python·Docker·포트(2181·9092·29092)를 점검하고 가상환경과 `kafka-python` 을 준비함

```
python scripts/setup_practice.py 4
```

```
cd practice/chapter3 && docker compose up -d zookeeper kafka
```

- 5·6절은 브로커 없이 실행됨
- 4주차 실습 중 중복 제거 부분은 1번 실행이 남긴 파일을 읽을 뿐이므로 Python 표준 라이브러리만 있으면 됨
- Docker를 못 쓰는 환경이면 5·6절만 직접 실행하고, 3·4절은 저장소에 들어 있는 산출물을 읽음
- 실행하기 전에 네 가지를 예측해 적음. 남이 만든 산출물을 먼저 열면 거기 적힌 숫자가 정답처럼 보여 "왜 이 값인가"를 묻지 않게 됨

표 4.1 예측표 — 산출물을 열기 전에 채운다

예측 항목	내가 적은 값	실제 값	어긋난 이유
200건을 지역구 8개 키로 파티션 3개에 보냈을 때, 가장 많이 받은 파티션의 건수			
커밋을 먼저 한 소비자가 10건 처리 후 죽었을 때 — 유실 / 중복	/	/	
처리를 먼저 한 소비자가 10건 처리 후 죽었을 때 — 유실 / 중복	/	/	
중복 10건이 섞인 40건을 UPSERT로 적재했을 때 — 행 수 / 총 관찰 수	/	/	

- 이 장에서 실행할 것은 둘이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	<code>run_kafka.py</code> (4-1·4-2 호출)	전달 방식 셋을 실제 브로커에서 돌려 유실·중복·지연을 기록	3·4	Docker + 3주차 compose의 zookeeper-kafka
2	<code>run_dedup.py</code> (4-3 호출)	1번이 남긴 중복 스트림을 저장 전략 넷으로 적재	5·6	없음 — Python 표준 라이브러리만

- 입력 자료에서 무엇이 실측이고 무엇이 구성값인지 먼저 구분함
- **구성값** — 민원 이벤트의 내용(지역구·유형·접수 시각)은 코드가 만든 시뮬레이션임. 개인정보가 들어 있는 실제 민원을 실습에 쓸 수 없기 때문임
- **실측** — 그 이벤트가 어느 파티션에 놓였는지, 몇 건이 유실·중복되었는지, 지연이 몇 밀리초였는지는 실제 브로커가 돌려준 값임
- 그러므로 이 장의 관찰 대상은 데이터의 내용이 아니라 **브로커의 전달 동작**임

3. 파티션이 순서를 지키는 범위를 정한다

- Kafka의 **토픽(topic)** 은 이벤트를 종류별로 담는 이름이고, 토픽은 하나 이상의 **파티션(partition)** 으로 나뉜다
- 파티션은 뒤에만 이벤트를 덧붙이는 로그이며, 이벤트마다 그 안에서의 위치 번호인 **오프셋(offset)** 이 붙음
- 오프셋은 파티션마다 따로 0부터 증가함. 그러므로 **순서도 토픽 전체가 아니라 파티션 안에서만 보장됨**
- 토픽을 하나의 줄 서기로 생각하면 틀림. 파티션 3개짜리 토픽은 창구가 셋인 민원실에 가까움
- 순서 보장 범위를 정하는 것은 **생산자(producer)** 가 붙이는 키다
- 같은 키를 가진 이벤트는 항상 같은 파티션으로 감. 키를 해시해 파티션 수로 나눈 나머지가 파티션 번호가 되기 때문임
- 지역구를 키로 쓰면 같은 지역구의 접수 순서가 보존되고, 민원 ID를 키로 쓰면 한 민원의 접수·배정·처리 순서가 보존됨
- 그래서 키 선택은 부하를 고르게 나누는 설정이 아니라 **무엇끼리 순서를 지킬지 정하는 업무 판단**이다
- 키를 정하면 부하 분포는 따라오는 결과이지 고를 수 있는 값이 아님
- 지역구를 키로 쓰는 순간 민원이 많은 지역구의 파티션이 더 많이 받는 것을 막을 수 없음
- 균등 분배를 원하면 키를 버려야 하고, 키를 버리면 순서 보장도 함께 버리는 것임
- 첫 번째 실행. 지역구 8개를 키로 200건을 파티션 3개에 보냄

```
cd practice/chapter4
python run_kafka.py
```

```
# 출처: practice/chapter4/data/output/ch4_produce_baseline.json
{
  "scenario": "baseline", "acks": "all", "sent": 200, "elapsed_sec": 0.301,
  "partition_distribution": { "0": 61, "1": 76, "2": 63 },
  "district_to_partitions": {
    "강남구": [1], "관악구": [0], "마포구": [1], "성동구": [0],
    "송파구": [2], "용산구": [2], "종로구": [2], "중구": [1]
  },
  "key_to_single_partition": true
}
```

□ 화면에 나온 값에서 읽을 것

- `key_to_single_partition: true` — 지역구 8개가 각각 파티션 하나에만 지정되었다는 뜻임. 같은 키가 여러 파티션에 흩어졌다면 이 값이 `false` 가 되고, 그 지역구의 접수 순서는 보장되지 않음
- **키가 2:3:3으로 나뉨** — 파티션 0에 관악구·성동구, 1에 강남구·마포구·중구, 2에 송파구·용산구·종로구가 들어갔음. 키 8개를 파티션 3개로 나누므로 애초에 똑같이 나뉠 수 없음
- **건수는 61·76·63** — 균등하게 나뉘었다면 200 나누기 3인 66.7건이었을 것임. 파티션 1이 76건으로 가장 많은데, 키를 3개 받았기 때문임
- **분포는 고를 수 있는 값이 아님** — 이 61·76·63은 설정으로 정한 값이 아니라 "지역구를 키로 쓴다"는 판단에서 따라 나온 결과임

- 파티션 수를 늘리면 지역구와 파티션의 대응이 끊긴다
- 파티션 번호는 키의 해시값을 파티션 수로 나눈 나머지이므로, 나누는 수가 3에서 4로 바뀌면 같은 지역구가 다른 파티션으로 이동한다.
- 이동한 시점을 경계로 그 지역구의 순서 보장이 끊기고, 옛 파티션에 남은 이벤트와 새 파티션에 쌓이는 이벤트의 앞뒤가 뒤섞인다.
- 그래서 파티션 증설은 성능 조정이 아니라 순서 보장 범위의 변경이며, 감사 대상 업무라면 변경 시점을 기록해 두어야 한다.

4. 커밋 한 줄의 위치가 유실과 중복을 가른다

- **소비자(consumer)** 는 자기가 어디까지 처리했는지를 **커밋된 오프셋**으로 브로커에 저장함
- 소비자가 죽었다 다시 살아나면 그 위치부터 다시 읽음. 2절의 책갈피가 이것임
- 그러므로 "커밋을 언제 하는가"가 장애가 났을 때 무엇이 벌어질지를 결정함
- 커밋을 처리보다 **먼저** 하면 **최대 한 번(at-most-once)** 전달이 됨
- 읽자마자 책갈피를 뒤로 옮겨 놓는 방식임
- 처리 도중 장애가 나면 이미 커밋된 이벤트는 다시 읽히지 않음 → 중복은 없고 **유실**이 생김

- 처리를 커밋보다 **먼저** 하면 **최소 한 번(at-least-once)** 전달이 됨
- 다 처리하고 나서 체크포인트를 옮기는 방식임
- 처리는 끝났는데 커밋 전에 장애가 나면 그 구간을 다시 읽음 → 유실은 없고 **중복**이 생김
- 실습 코드에서 이 선택은 `commit()` 호출을 처리 루프 앞에 두느냐 뒤에 두느냐가 전부임

```
# practice/chapter4/code/4-2-kafka-consumer.py 발췌
if args.mode == "at-most-once":
    consumer.commit()           # 처리 "전" 커밋 → 처리 중 장애면 그 배치는 유실
    for tp, records in batch.items():
        for rec in records:
            meta = build_meta(tp, rec)
            process_event(rec.value, meta, store_path)
if args.mode == "at-least-once":
    consumer.commit()           # 처리 "후" 커밋 → 커밋 전 장애면 그 배치는 재수신
```

- 자동 커밋은 이 선택을 코드에서 빼앗아 간다
- `enable_auto_commit` 을 켜 두면 일정 시간마다 브로커가 알아서 오프셋을 옮기므로, 저장에 끝나기 전에 체크포인트가 넘어갈 수 있음
- 실습에서는 `enable_auto_commit=False` 로 꺼 두고 커밋 시점을 코드가 정함
- 두 번째 실행. 같은 명령이 세 시나리오를 차례로 돌림 — 정상 경로, 커밋 먼저 + 장애, 처리 먼저 + 장애

```
python run_kafka.py
```

```
# 출처: practice/chapter4/data/output/ch4_delivery_report.json
"A_baseline": { "produced": 200, "processed_records": 200, "unique_events": 200,
                "duplicate_records": 0, "lost_events": 0,
                "latency_ms": { "count": 200, "mean": 3.5, "max": 110.7 } }
"B_at_most_once_crash": { "produced": 30, "crash_after": 10, "crash_exit_code": 42,
                           "processed_records": 10, "unique_events": 10,
                           "duplicate_records": 0, "lost_events": 20 }
"C_at_least_once_crash": { "produced": 30, "crash_after": 10, "crash_exit_code": 42,
                            "processed_records": 40, "unique_events": 30,
                            "duplicate_records": 10, "lost_events": 0 }
```

```
# 출처: practice/chapter4/data/output/ch4_consume_g-amo.json, ch4_consume_g-alo.json
g-amo (at-most-once) 재시작한 소비자의 processed_this_run: 0
g-alo (at-least-once) 재시작한 소비자의 processed_this_run: 30
```

□ 화면에 나온 값에서 읽을 것

○ **정상 경로에서는 둘 다 0** — 200건을 보내 200건을 처리했고 유실도 중복도 없음. 장애가 없으면 두 방식의 차이가 드러나지 않으므로, 차이를 보려면 장애를 일부러 만들어야 함

○ **커밋 먼저: 20건이 사라짐** — 30건을 커밋해 둔 뒤 10건만 처리하고 죽었음. 재시작한 소비자는 책갈피가 이미 끝에 가 있으므로 **0건**을 읽고 끝냄(`processed_this_run: 0`). 나머지 20건은 어디에도 남지 않음

○ **처리 먼저: 10건이 두 번 처리됨** — 10건을 처리하고 커밋 전에 죽었음. 재시작한 소비자는 **30건을 처음부터** 다시 읽어(`processed_this_run: 30`) 처리 기록이 $10 + 30 = 40$ 건이 되었고, 고유 이벤트는 30건이므로 중복이 10건임

○ **두 숫자의 성격이 다름** — 중복 10건은 나중에 지울 수 있지만, 유실 20건은 원본이 브로커의 보존 기간 안에 남아 있지 않는 한 되살릴 수 없음

○ **지연은 평균 3.5ms, 최대 110.7ms** — 정상 경로에서 생산부터 소비까지 걸린 시간임. 이 값은 소비자를 먼저 띄워 놓고 생산했을 때의 값이며, 소비자를 나중에 시작하면 지연을 지배하는 것은 브로커가 아니라 소비 시작 시점이 됨

□ 그래서 공공 데이터의 기본값은 **at-least-once**다

○ 교정할 수 있는 중복보다 복구할 수 없는 유실을 먼저 피함

○ 재난 신고나 민원 접수에서 20건이 조용히 사라지는 쪽과, 10건이 두 번 들어와 나중에 걸러지는 쪽 중 무엇이 더 위험한지는 비교할 필요가 없음

○ 대신 중복을 흡수할 책임이 하류 저장소로 넘어감. 그것이 5절임

- 유실 20건은 오류 로그를 남기지 않았다
- 종료 코드 42는 실습이 장애를 흉내 내려고 일부러 낸 값이고, 재시작한 소비자는 읽을 것이 없다고 판단해 정상적으로 끝났다.
- 1주차의 침묵 실패가 수집 계층에서 다시 나타나는 자리다. 유실을 찾은 방법은 로그를 뒤진 것이 아니라 **생산 30건과 고유 처리 10건을 맞춰 본 것**이다.
- 그러므로 전달 보증을 다루는 시스템에는 생산 건수와 처리 건수를 주기적으로 대사하는 절차가 함께 있어야 한다(1주차에서 지표로 만든다).

5. 중복은 상류에서 못 막고 하류 저장소에서 흡수한다

□ 4절이 만든 중복 10건을 그대로 입력으로 씀

○ `data/input/ch4_alo_duplicate_stream.jsonl` 은 시나리오 C가 실제 브로커에서 남긴 처리 기록 40건의 스냅샷임

○ 중복을 흉내 낸 자료가 아니라 브로커 장애 실험이 실제로 만든 중복임

□ 반복 전달을 판정하려면 **중복 제거 키(deduplication key)** 가 있어야 함

○ 이벤트가 처음 만들어질 때 붙인 고유 ID여야 함. 실습에서는 생산자가 `alo-000003` 같은 `event_id` 를 만들어 붙임

- 전송 함수 안에서 매번 새 ID를 만들면 재전송된 같은 이벤트가 서로 다른 이벤트로 보여 중복 제거가 성립하지 않음
- 기술적 반복 전달과, 시민이 실제로 다시 접수한 별개 민원은 구분해야 함. 후자는 지워야 할 중복이 아님
- 애플리케이션에서 "있는지 조회한 뒤 없으면 저장"하는 방식은 경쟁 조건에 취약함
- 소비자 두 대가 거의 동시에 조회하면 둘 다 "없다"고 판단해 둘 다 삽입함
- 저장소의 유니크 제약은 한 위치에서 원자적으로 두 번째 삽입을 막으므로 이 틈이 생기지 않음
- 충돌이 났을 때 무엇을 할지는 저장 목적이 정함
- **실패** — 중복 도착 자체가 오류인 경우에 씀
- **무시(INSERT OR IGNORE)** — 업무 상태는 지키지만 중복이 있었다는 기록도 함께 사라짐
- **UPSERT(ON CONFLICT DO UPDATE)** — 행 하나를 유지하면서 몇 번 관찰했는지와 마지막 관찰 시각을 갱신함

```
INSERT INTO complaints_upsert VALUES (?, ?, ?, ?, ?, 1)
ON CONFLICT(event_id) DO UPDATE SET
    last_seen_at = excluded.last_seen_at, seen_count = seen_count + 1
```

□ 세 번째 실행. Docker 없이 실행됨

```
cd practice/chapter4
python run_dedup.py
```

출처: practice/chapter4/results/4-3-deduplicate-events.log

[1] 입력 - 처리 기록 40건, 고유 event_id 30개

출처: data\input\ch4_alo_duplicate_stream.jsonl

두 번 나타난 event_id 10개

alo-000003 alo-000007 alo-000008 alo-000009 alo-000010 alo-000012 alo-000014 alo-000022 alo-000027 alo-000030

예) alo-000003 partition 0 offset 0 - 같은 위치가 두 번 처리됨

1차 2026-07-07T01:59:20.513309+00:00

2차 2026-07-07T01:59:35.717570+00:00 (간격 15.2초)

[2] 저장 전략별 결과

단계	행 수	관찰 지표
A 제약 없는 INSERT	40	중복 행 10
B UNIQUE + OR IGNORE	30	무시된 중복 10
C UPSERT DO UPDATE	30	총 관찰 40 / 재관찰 이벤트 10

□ 화면에 나온 값에서 읽을 것

○ 중복된 10개는 크래시 전에 처리한 바로 그 10건임 — `alo-000003` 부터 `alo-000030` 까지의 목록이 4절 시나리오 C에서 커밋 전에 처리하고 죽은 건들임. 중복은 아무 데서나 생기지 않고 **마지막 커밋과 장애 사이의 구간에서** 생김

○ 같은 파티션·같은 오프셋이 두 번 나타남 — `alo-000003` 은 둘 다 partition 0의 offset 0임. 두 번째 처리가 새 이벤트를 읽은 것이 아니라 **같은 위치를 다시 읽은 것**임을 오프셋이 증명함

○ 두 처리 시각의 간격이 15.2초 — 크래시한 소비자를 브로커가 그룹에서 빼기까지 기다리는 시간(세션 타임아웃 10초)과 재시작 대기를 합친 값임. 재수신은 즉시가 아니라 이만큼 늦게 옴

○ 제약이 없으면 40행, 유니크 제약이 있으면 30행 — 상류에서 중복이 흘러와도 저장소에 제약 한 줄이 있으면 업무 상태는 30건으로 지켜짐. 반대로 제약이 없으면 상류 중복이 그대로 통계가 됨

- 중복 10건은 "잘못된 데이터"가 아니다
- 재수신된 10건은 내용이 정확하고 오프셋도 정상이다. 다만 **같은 사실을 두 번 말한 것**이다.
- 그러므로 판정 기준은 데이터의 정확성이 아니라 동일성이고, 동일성을 판정하는 것이 중복 제거 키다.
- 키를 잘못 고르면 정상 민원 두 건이 같은 이벤트로 합쳐진다. 접수 시각이나 지역구를 키로 삼지 않는 이유다.

6. 멍등한 반영은 업무 상태 지문으로 확인한다

□ 여러 번 전달되어도 업무 상태가 한 번 반영된 것과 같으면 그 처리를 **멍등(idempotent)** 하다고 한다

○ 5절의 B와 C가 그런 처리임. 같은 이벤트가 두 번 와도 행은 하나임

○ 다만 멍등성을 "행 수가 안 늘었다"로만 확인하면 부족함. 행 안의 값이 조용히 덮어써졌을 수 있기 때문임

□ 그래서 실습은 저장된 내용을 **업무 상태와 전달 이력** 둘로 나눔

○ 업무 상태 — `event_id · partition · offset · first_seen_at`. 같은 스트림을 몇 번 흘리든 변하면 안 되는 값임

○ 전달 이력 — `seen_count · last_seen_at`. 몇 번 받았는지를 적는 값이므로 다시 흘리면 **늘어나는 것이 정상**임

○ 업무 상태 네 칼럼만 모아 해시를 내면 16자리 **지문**이 나오고, 두 시점의 지문을 비교해 불변을 한 줄로 보일 수 있음

□ D단계는 같은 40건을 한 번 더 통째로 흘려보냄

○ 실제로 이런 일이 벌어지는 경우가 세 가지임 — 파이프라인을 고친 뒤의 재처리, 3주차에서 본 리플레이, 배치 작업의 재시도

○ 멍등하지 않은 파이프라인은 이 세 경우마다 통계가 부풀어 오름

□ 5절과 같은 명령의 뒤쪽 출력. 같은 스트림을 한 번 더 투입한 결과임

```
# 출처: practice/chapter4/results/4-3-deduplicate-events.log
```

[2] 저장 전략별 결과

단계	행 수	관찰 지표
C UPSERT DO UPDATE	30	총 관찰 40 / 재관찰 이벤트 10
D C에 같은 스트림 재투입	30	총 관찰 80 / 재관찰 이벤트 30

[3] 업무 상태 지문 - event_id.partition.offset.first_seen_at

C eed1522a7ca844bd
D eed1522a7ca844bd

일치: 예 - 총 관찰은 40에서 80으로 늘었지만 행 수는 30으로 그대로임

□ 화면에 나온 값에서 읽을 것

- 지문이 같으면 재처리해도 안전함 — C와 D의 지문이 둘 다 eed1522a7ca844bd 임. "다시 넣어도 괜찮았다"가 아니라 같은 값이라는 사실을 한 줄로 보임
- 총 관찰이 40에서 80으로 늘어난 것은 오류가 아님 — 40건을 한 번 더 흘렸으니 관찰 수는 두 배가 되는 것이 맞음. 전달 이력은 늘어나야 하는 값임
- 재관찰 이벤트가 10에서 30으로 늘어남 — C에서는 10건만 두 번 관찰됐지만, D에서 전부 한 번씩 더 받았으므로 30건 모두 재관찰 상태가 됨. 이 값도 늘어나는 것이 정상임
- 두 축을 나누지 않으면 판정이 불가능함 — 모든 칼럼을 한 해시에 넣었다면 D에서 지문이 달라졌을 것이고, 정상인 시스템을 고장으로 판정했을 것임

□ 여기까지가 공공 파이프라인에서 쓰는 실제 답임

- 전달은 at-least-once로 두어 유실을 막고, 반영은 유니크 제약과 UPSERT로 멍등하게 만들어 중복을 흡수함
- 확인은 "중복이 없었다"가 아니라 "중복이 있었지만 업무 상태 지문이 그대로다"로 함

- Kafka의 exactly-once는 이 문제를 대신 풀어 주지 않는다
- exactly-once는 Kafka 안에서 읽고 쓰는 구간의 보장이지, 외부 데이터베이스에 반영한 결과까지 보장하지 않는다.
- 더 근본적으로, 저장 요청을 보낸 뒤 응답이 오지 않으면 기록이 실패한 것인지 응답만 유실된 것인지 구분할 수 없다. 보낸 쪽이 할 수 있는 일은 다시 보내는 것뿐이다.
- 그래서 전달 보장과 반영 보장을 나누어 생각하고, 반영 쪽은 저장소 제약과 멍등 쓰기로 직접 만든다.

7. 핵심 요약

- 1. 순서는 토픽이 아니라 파티션 안에서만 보장되고, 그 범위를 정하는 것은 키다. (3절)
 - 지역구 8개를 키로 200건을 보내자 키가 파티션 3개에 2:3:3으로 나뉘고 건수가 61·76·63이 되었으며, `key_to_single_partition` 이 `true` 로 나와 같은 지역구가 한 파티션에만 들어갔음이 확인됨
 - 분포는 고를 수 있는 값이 아니라 키 선택에서 따라 나오는 결과임. 균등 분배를 얻으려면 키를 버려야 하고, 그러면 순서 보장도 함께 버리게 됨
- 2. 커밋을 처리보다 먼저 하면 유실, 나중에 하면 중복이 생긴다. (4절)
 - 같은 30건·같은 장애 시점인데 커밋을 먼저 한 쪽은 유실 20건, 나중에 한 쪽은 중복 10건이 나왔음
 - 재시작한 소비자가 읽은 건수가 각각 0건과 30건으로 갈렸고, 둘 다 오류 없이 끝났음. 유실은 로그가 아니라 생산 건수와 고유 처리 건수의 대사로 발견함
- 3. 공공 데이터는 교정할 수 있는 중복보다 복구할 수 없는 유실을 먼저 피한다. (4절)
 - 기본 경로를 `at-least-once`로 두고, 중복을 흡수할 책임을 하류 저장소로 넘김
 - 그 대가로 자동 커밋을 끄고 커밋 시점을 코드가 직접 정해야 함
- 4. 중복 제거 키와 유니크 제약이 상류 중복을 업무 상태에서 걸러 낸다. (5절)
 - 같은 40건을 제약 없이 넣으면 40행, 유니크 제약을 걸면 30행이 되었고, 중복된 10건은 모두 마지막 커밋과 장애 사이 구간의 이벤트였음
 - 같은 파티션·같은 오프셋이 두 번 나타난 것이 재수신의 증거이며, 두 처리 시각의 간격은 15.2초였음
- 5. 멍등성은 업무 상태와 전달 이력을 나누어야 검증된다. (6절)
 - 같은 스트림을 한 번 더 흘리자 총 관찰은 40에서 80으로 늘었지만 행 수는 30, 업무 상태 지문은 `eed1522a7ca844bd` 로 그대로였음
 - 두 축을 한 해시에 섞었다면 정상인 시스템을 고장으로 판정했을 것임

8. 과제

- 실습을 순서대로 실행해 만든 산출물을 LMS에 제출한다
 - 1. `practice/chapter4/data/output/ch4_delivery_report.json` — Docker를 쓸 수 없었다면 저장소에 들어 있는 파일을 그대로 제출하고, 실행이 어디서 막혔는지 적는다.
 - 2. `practice/chapter4/data/output/ch4_dedup_report.json`
 - 3. `practice/chapter4/results/4-3-deduplicate-events.log`
 - 4. 표 4.1 예측표 — 실제 값과 대조해 채운 최종본. 형식은 무방하다.
- 이어서 네 가지를 각각 세 문장 이내로 쓴다
 - 1. 예측한 네 항목과 실제 값을 나란히 적고, 어긋난 항목마다 자기 예측의 어느 전제가 틀렸는지 밝힌다.
 - 2. 시나리오 B의 유실 건수와 C의 중복 건수를 옮겨 적고, 두 값이 갈린 원인을 `commit()` 한 줄의 위치로 설명한다.
 - 3. 로그 [1] 에 나온 중복 `event_id` 10개가 왜 하필 그 10개인지, 4절의 크래시 시점과 연결해 설명한다.

4. C와 D의 업무 상태 지문은 같은데 총 관찰 수는 40과 80으로 다르다. 이 차이가 왜 오류가 아닌지 업무 상태와 전달 이력으로 나누어 설명한다.

- ☐ 값을 지어내지 않는다
- ☐ 실행이 안 되면 어디서 막혔는지 그대로 쓴다
- ☐ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

- ☐ 실습을 더 해 보려면 `practice/chapter4/` 에서 다음을 시도함
 - `4-1-kafka-producer.py` 에 `--partitions 4` 를 주고 다시 보내, `district_to_partitions` 에서 어느 지역구가 다른 파티션으로 옮겨 갔는지 확인함
 - `4-2-kafka-consumer.py` 의 `--crash-after` 값을 바꿔, 장애 시점이 유실·중복 건수를 어떻게 바꾸는지 확인함
 - `4-3-deduplicate-events.py` 의 `upsert_stats()` 에서 지문 계산에 `seen_count` 를 포함시킨 뒤, D단계에서 지문이 달라지는 것을 확인함
 - `4-1-kafka-producer.py` 의 `acks="all"` 을 `"1"` 이나 `"0"` 으로 고쳐, `elapsed_sec` 이 어떻게 달라지는지 재보
- ☐ 뒤 주차에서 이어지는 내용
 - 이 장에서 모은 이벤트를 시간 창으로 묶고 늦게 도착한 이벤트를 처리하는 방법은 5주차에서 실행함
 - 같은 데이터를 배치로 다시 처리해 확정치를 만드는 방법은 6주차에서 다룸
 - 생산 건수와 처리 건수의 대사를 지표로 만들어 침묵 실패를 잡는 방법은 11주차에서 다룸
- ☐ 원문을 직접 보고 싶다면 다음 두 곳이 출발점임
 - Apache Kafka Documentation, 「Message Delivery Semantics」 — at-most-once·at-least-once·exactly-once의 공식 정의. <https://kafka.apache.org/documentation/#semantics>
 - Apache Kafka Documentation, 「Consumers」 — 컨슈머 그룹과 오프셋 관리. https://kafka.apache.org/documentation/#intro_consumers